ONLINE MASTERS IN **DATA SCIENCE**

DSC258: NATURAL LANGUAGE PROCESSING

BUILDING LINEAR CLASSIFIERS

UC San Diego

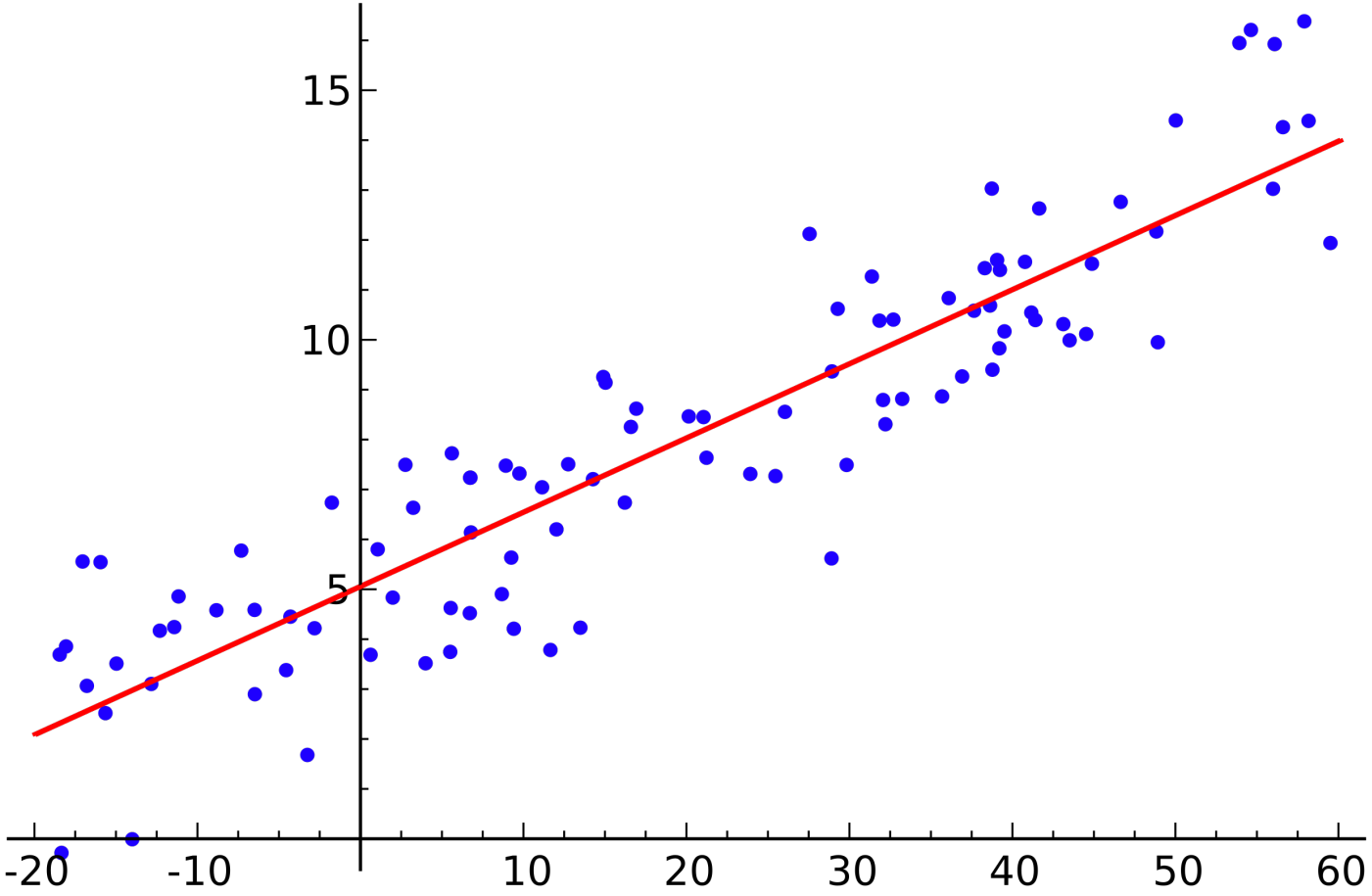
COMPUTER SCIENCE & ENGINEERING

HALICIOĞLU DATA SCIENCE INSTITUTE

Bag-of-words Text Classifier

- What's text classification? How to evaluate a classifier?
- Bag-of-words Representations
- **Building a linear classifier upon bag-of-words**

- Let’s start with Ordinary Least Square (OLS) Regression



OLS: Example

- The following data set gives average heights and weights for American women aged 30–39 (source: *The World Almanac and Book of Facts, 1975*).

Height (m)	1.47	1.50	1.52	1.55	1.57	1.60	1.63	1.65	1.68	1.70	1.73	1.75	1.78	1.80	1.83
Weight (kg)	52.21	53.12	54.48	55.84	57.20	58.57	59.93	61.29	63.11	64.47	66.28	68.10	69.92	72.19	74.46

- Build a relation between Weight and Height
  - Weight = *f* (Height)

OLS: Linear Prediction

- Linear Prediction for the i-th instance

$$\hat{\boldsymbol{y}}_{\boldsymbol{i}} = \sum_{j=1}^{d} \boldsymbol{x}_{i,j} \boldsymbol{w}_j + b$$

- $\hat{\boldsymbol{y}}_{\boldsymbol{i}}$ is the prediction
- $\boldsymbol{x}_i$ is the feature vector of the i-th instance
- d is the number of dimensions of the feature vector

- $\boldsymbol{w}$ and b are parameters to learn

OLS: Square Loss Function

- Square Loss for the i-th instance

$$l(\boldsymbol{y}_i, \widehat{\boldsymbol{y}}_i) = (\boldsymbol{y}_i - \widehat{\boldsymbol{y}}_i)^2$$

- Square loss for all ***n*** instances

$$L = \sum_{i=1}^{n} l(\boldsymbol{y}_i, \widehat{\boldsymbol{y}}_i) = \sum_{i=1}^{n} (\boldsymbol{y}_i - \widehat{\boldsymbol{y}}_i)^2$$

OLS via sklearn

sklearn.linear_model.LinearRegression

class `sklearn.linear_model.` **`LinearRegression`**(*fit_intercept=True*, *normalize=False*, *copy_X=True*, *n_jobs=None*) [source]

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients w = (w1, ..., wp) to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

- The model minimize the square loss without any regularization
- n_jobs → # of workers, mainly to speed up the training process

- Logistic Prediction for the i-th instance

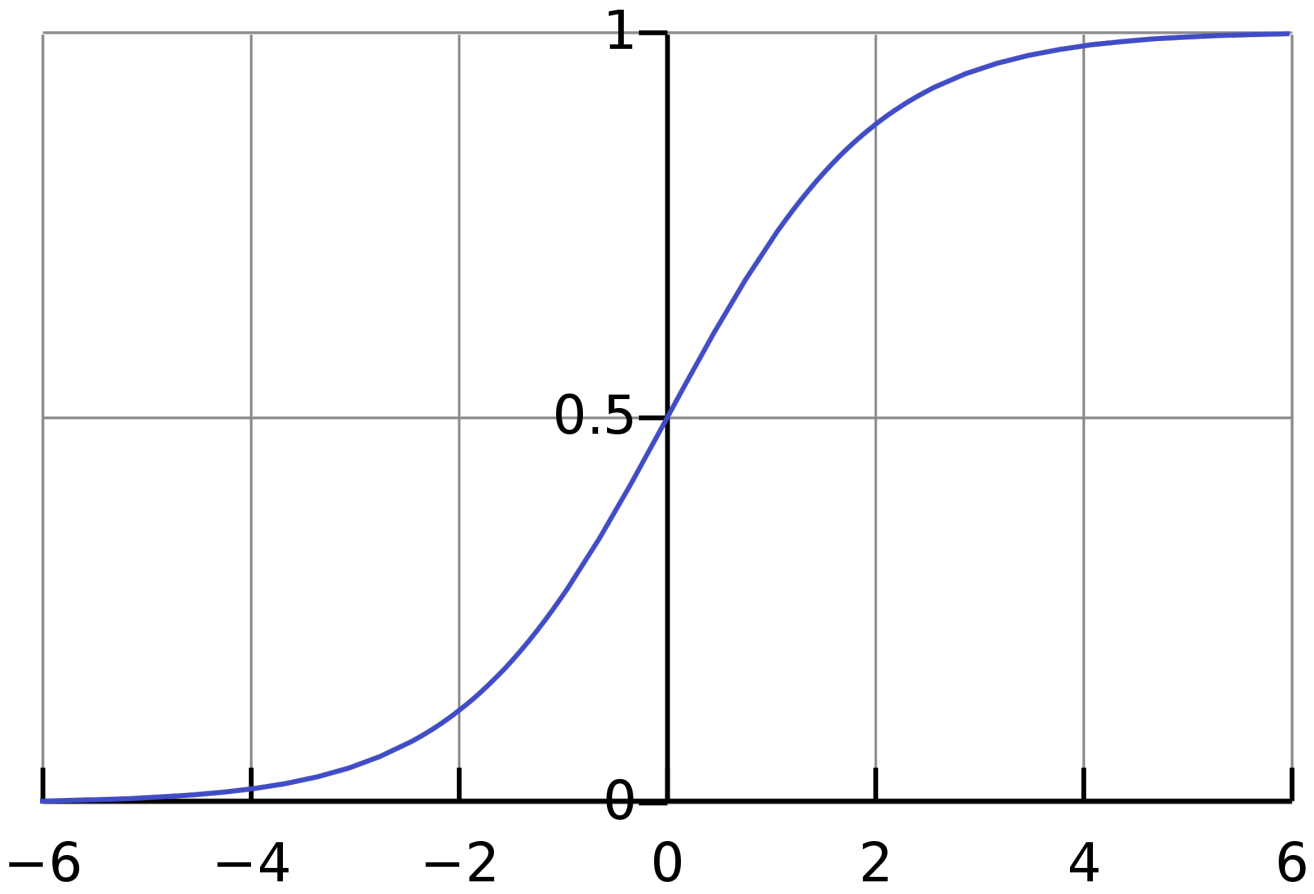
$$\hat{\boldsymbol{y}}_{\boldsymbol{i}} = \sigma\left(\sum_{j=1}^{d} \boldsymbol{x}_{i,j}\boldsymbol{w}_j + b\right)$$

- $\hat{\boldsymbol{y}}_{\boldsymbol{i}}$ is the prediction
- $\boldsymbol{x}_i$ is the feature vector of the i-th instance
- d is the number of dimensions of the feature vector
- σ is the sigmoid function

- $\boldsymbol{w}$ and b are parameters to learn

Sigmoid Function

$$\sigma(x) = \frac{e^x}{e^x + 1} = \frac{1}{1 + e^{-x}}$$



Logistic Loss Function

- Some facts
  - $\boldsymbol{y}_i \in \{0, 1\}$
  - $\hat{\boldsymbol{y}}_i \in [0, 1]$

- Logistic Loss for the i-th instance

$$l(\boldsymbol{y}_i, \hat{\boldsymbol{y}}_i) = -(y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$$

Using logistic regression for regression

- Apply min-max normalization for the target variable
  - $\frac{x - min}{\max - min}$ → [0, 1]
  - Could be other variants

- And then, revert it back for predictions/evaluations

- It works for bounded regression!

Logistic regression via sklearn

sklearn.linear_model.LogisticRegression

class `sklearn.linear_model.` **`LogisticRegression`**(*penalty='l2', dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0, warm_start=False, n_jobs=None, l1_ratio=None*) ¶ [source]

- We first split the entire X, y dataset into train and test

In [64]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1, test_size=0.2, shuffle=True)

print(len(X_train), len(X_test))
```

```
40000 10000
```

Logistic regression for sentiment analysis

- We first split the entire X, y dataset into train and test

```
In [64]: from sklearn.model_selection import train_test_split

         X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1, test_size=0.2, shuffle=True)

         print(len(X_train), len(X_test))

         40000 10000
```

- Build a model on (a subset of) training set, and then evaluate on the test set

```
In [68]: # train a logistic regression model
         from sklearn.linear_model import LogisticRegression

         clf = LogisticRegression(random_state=0).fit(X_train[:1000], y_train[:1000])

         clf.score(X_test, y_test)

Out[68]: 0.8255
```

- Sparse matrix needed to build models on the entire training set

SUMMARY

- What’s text classification? How to evaluate a classifier?
- Bag-of-words Representations
- Building a linear classifier upon bag-of-words

Feel free to ask questions in Office Hours / Piazza